Web Application Security Assessment Report

Candidate Information

Name: E.K. Sanali Akithma Senarathna

Email: sanalisenarathna03@gmail.com

Contact Number: +94 76 881 9661

Assessment Information

Position Applied: Intern – Cyber Security

Assessment Type: Web Application Penetration Testing

Date: 20/03/2026

Target and Environment

Target Application: OWASP Juice Shop

Testing Environment: Localhost (http://localhost:3000)

Operating System: Kali Linux (Virtual Machine)

Testing Tool: Burp Suite Community Edition

1. Introduction

This report presents the results of a security assessment conducted on a deliberately vulnerable web application which is OWASP Juice Shop. The purpose of this assessment was to identify, analyze, and exploit common web application vulnerabilities in a controlled environment.

The testing was performed using industry-standard tools and techniques to simulate real-world attack scenarios. The objective was not only to discover vulnerabilities but also to understand their impact and demonstrate how they could be exploited by an attacker. The findings in this report highlight security weaknesses in areas such as input validation, authentication, authorization, and configuration.

2. Scope

The scope of this assessment was limited to locally deployed instance of the OWASP Juice Shop application. Testing was conducted in a controlled environment to ensure no impact on external systems.

- **Application**: OWASP Juice Shop
- **Environment**: Localhost (http://localhost:3000)
- **Testing Type**: Black-box penetration testing
- **User Roles Tested**: Unauthenticated users, normal users, and administrative access scenarios

The assessment focused on identifying vulnerabilities within the application's web interface and API endpoints. No testing was performed on external services or infrastructure.

3. Methodology

The security assessment followed a structured approach based on common penetration testing practices and aligned with the OWASP Top 10 vulnerability categories.

The testing process included the following steps:

1. **Reconnaissance:**
   Initial exploration of the application to understand its structure, functionality, and available endpoints.
2. **Interception and Analysis:**
   HTTP requests and responses were intercepted using Burp Suite to analyze how the application processes user input and communicates with the server.
3. **Input Manipulation:**
   Various payloads were injected into input fields and API requests to test for vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS).
4. **Authentication and Authorization Testing:**
   Login mechanisms and access controls were tested to identify weaknesses such as authentication bypass and unauthorized access to restricted resources.
5. **Manual Testing of API Endpoints:**
   API endpoints were manually modified and tested to detect issues like Insecure Direct Object Reference (IDOR) and broken access control.
6. **Response Analysis:**
   Server responses were carefully analyzed to identify error messages, data leakage, and misconfigurations.
7. **Vulnerability Mapping:**
   Identified issues were categorized based on OWASP Top 10 standards to ensure proper classification and reporting.

This methodology ensured a comprehensive evaluation of the application's security posture while simulating realistic attack scenarios.

4. Findings

The security assessment identified multiple vulnerabilities affecting the application. These vulnerabilities range from critical issues such as SQL injection to misconfigurations and improper access control. Each finding is described below with its impact and recommend mitigation.

4.1 SQL Injection

Description

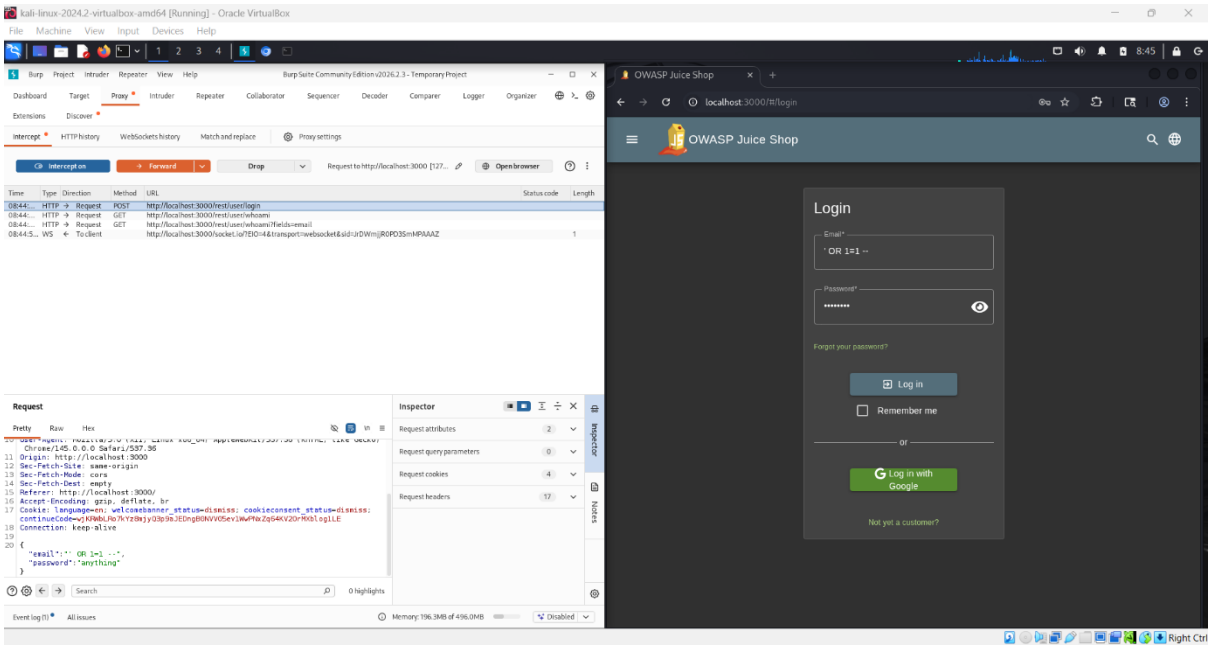
Login functionality is vulnerable to SQL Injection, allowing an attacker to bypass authentication mechanisms by manipulating input fields.

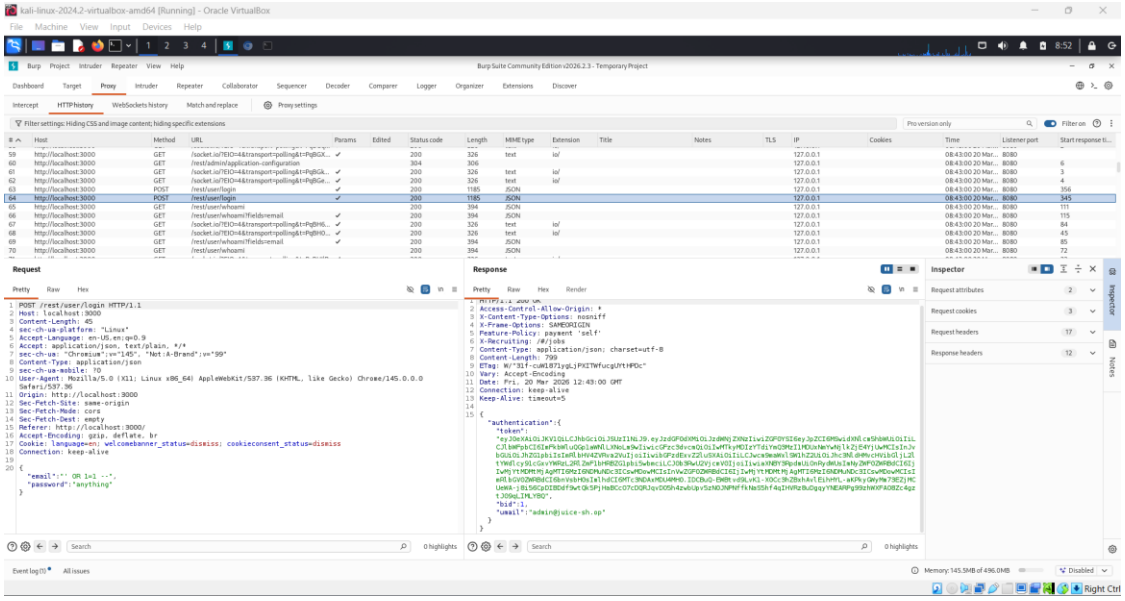
Steps to Reproduce

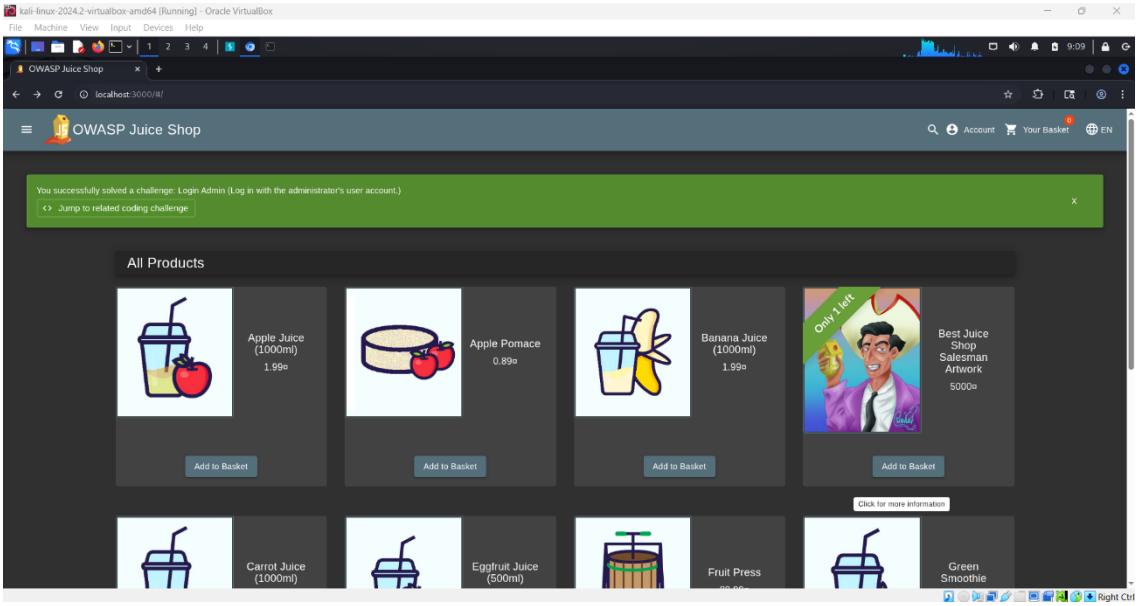
1. Open a web browser (Chromium) and navigate to the application's login page (http://localhost:3000/#/login).
2. On the login page locate the Email and Password input fields.
3. In the Email field, enter the following SQL injection payload exactly as shown:
   ' OR 1=1 –
4. In the Password field enter any random value.
5. Click the login button to submit the form.
6. Observe the behavior of the application after submission.

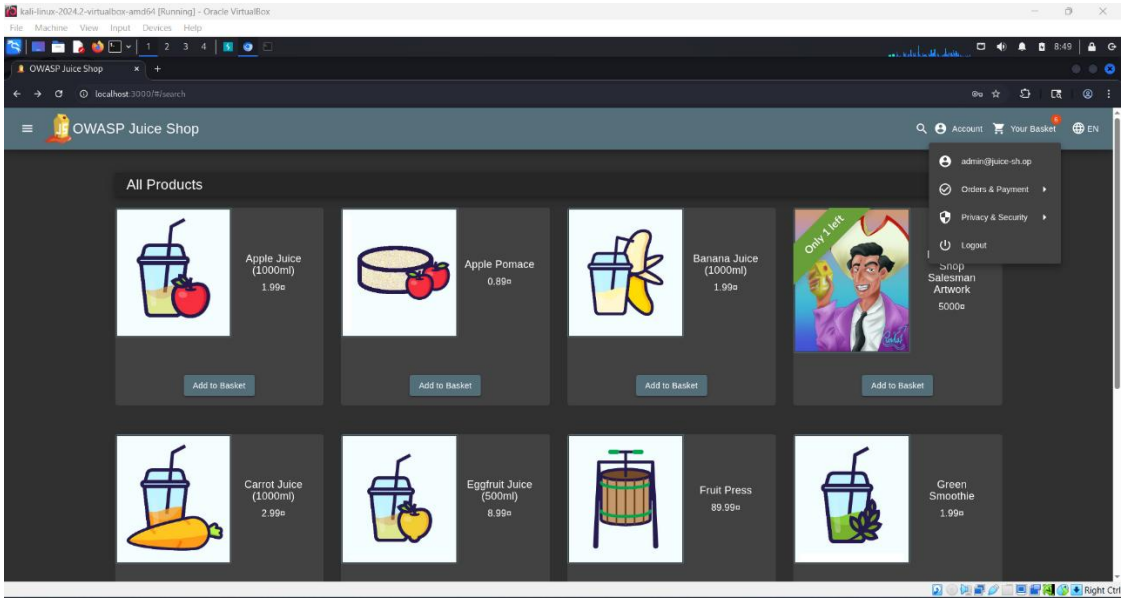
Proof of Concept

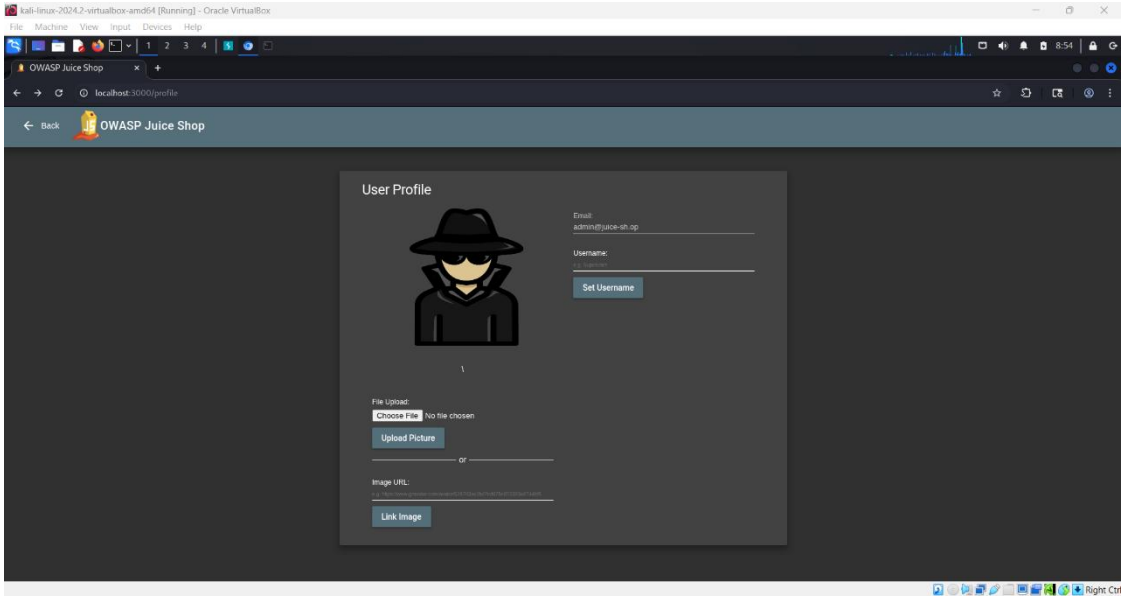
The application grants access without validating legitimate credentials, allowing login as an administrator.




kali-linux-2024.2-virtualbox-amd64 [Running] - Oracle VirtualBox
Burp Suite Community Edition v2026.2.3 - Temporary Project
Request
Response
Inspector


kali-linux-2024.2-virtualbox-amd64 [Running] - Oracle VirtualBox
OWASP Juice Shop
localhost:3000/#/
Account
Your Basket
EN
You successfully solved a challenge: Login Admin (Log in with the administrator's user account.)
Jump to related coding challenge
All Products
Apple Juice (1000ml) 1.99¤
Apple Pomace 0.89¤
Banana Juice (1000ml) 1.99¤
Only 1 left
Best Juice Shop Salesman Artwork 5000¤
Add to Basket
Click for more information
Carrot Juice (1000ml)
Eggfruit Juice (500ml)
Fruit Press
Green Smoothie


kali-linux-2024.2-virtualbox-amd64 [Running] - Oracle VirtualBox
OWASP Juice Shop
localhost:3000/#/search
Account
Your Basket
EN
admin@juice-sh.op
Orders & Payment
Privacy & Security
Logout
All Products
Apple Juice (1000ml) 1.99¤
Apple Pomace 0.89¤
Banana Juice (1000ml) 1.99¤
Only 1 left
Salesman Artwork 5000¤
Add to Basket
Carrot Juice (1000ml) 2.99¤
Eggfruit Juice (500ml) 8.99¤
Fruit Press 89.99¤
Green Smoothie 1.99¤



Impact

- Unauthorized access to user accounts
- Potential full system compromise
- Exposure of sensitive data

Severity

Critical

Real-World Attack Scenario

- An attacker could exploit the login form using SQL Injection to bypass authentication and gain administrative access. Once logged in, the attacker could view, modify, or delete sensitive data, potentially compromising the entire system.

Mitigation

- Use parameterized queries or prepared statements
- Implement strict input validation
- Avoid dynamic SQL queries

4.2 Cross-Site Scripting (XSS)

Description

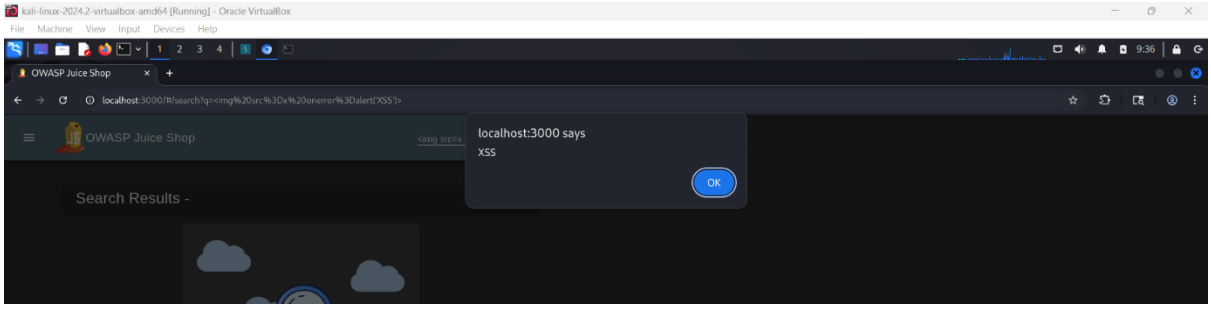
The application is vulnerable to reflected Cross-Site Scripting (XSS), allowing execution of arbitrary JavaScript in the user's browser.

Steps to Reproduce

1. Open a web browser and navigate to the application.
2. Locate the search functionality like search bar or search input within the application interface.
3. In the search input field, enter the following payload exactly as shown:
   <img src=x onerror=alert('XSS')>
4. Submit the search request by clicking the search button or pressing enter.
5. Observe the behavior of the application after submitting the input.

Proof of Concept

A JavaScript alert is executed in the browser, confirming successful injection.



Impact

- Session hijacking
- Execution of malicious scripts
- Redirection to malicious websites

Severity

High

Real-World Attack Scenario

- An attacker could inject malicious JavaScript into a vulnerable input field, which executes in other users' browsers. This could allow the attacker to steal session cookies, hijack user accounts, or redirect users to malicious phishing websites.

Mitigation

- Sanitize and validate all user inputs
- Encode output before rendering
- Implement Content Security Policy (CSP)

4.3 Insecure Direct Object Reference (IDOR)

Description

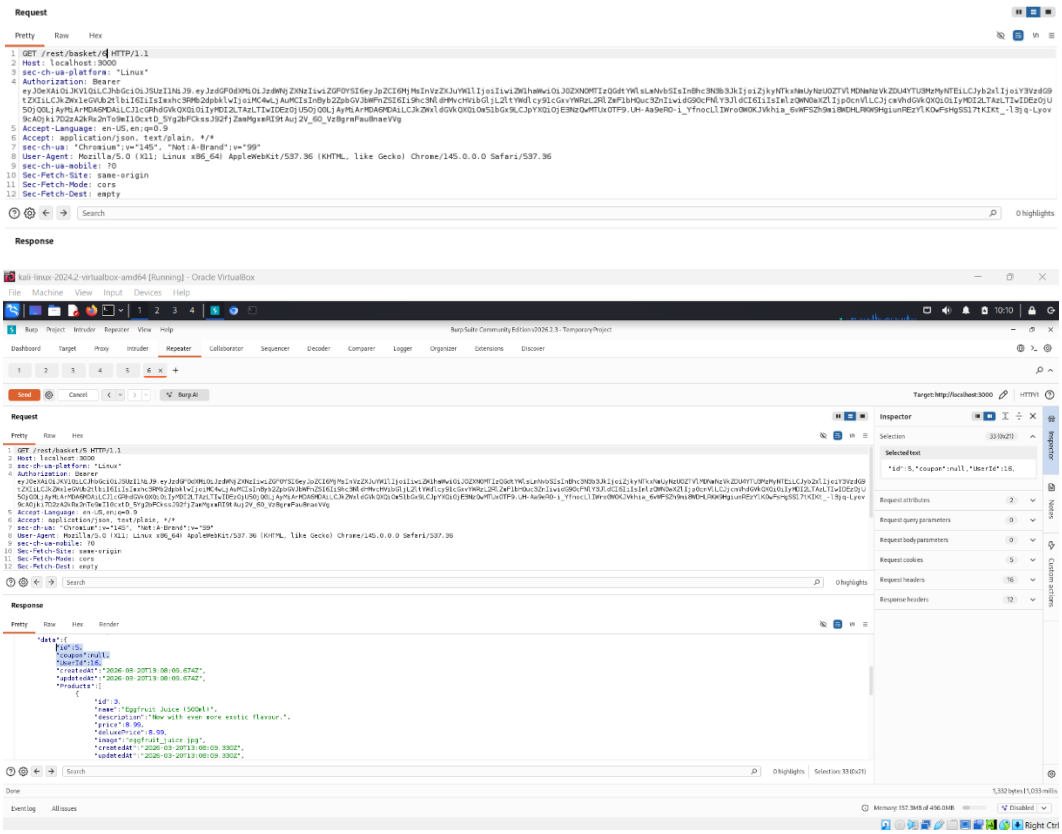
The application exposes object identifiers in API endpoints without proper authorization checks.

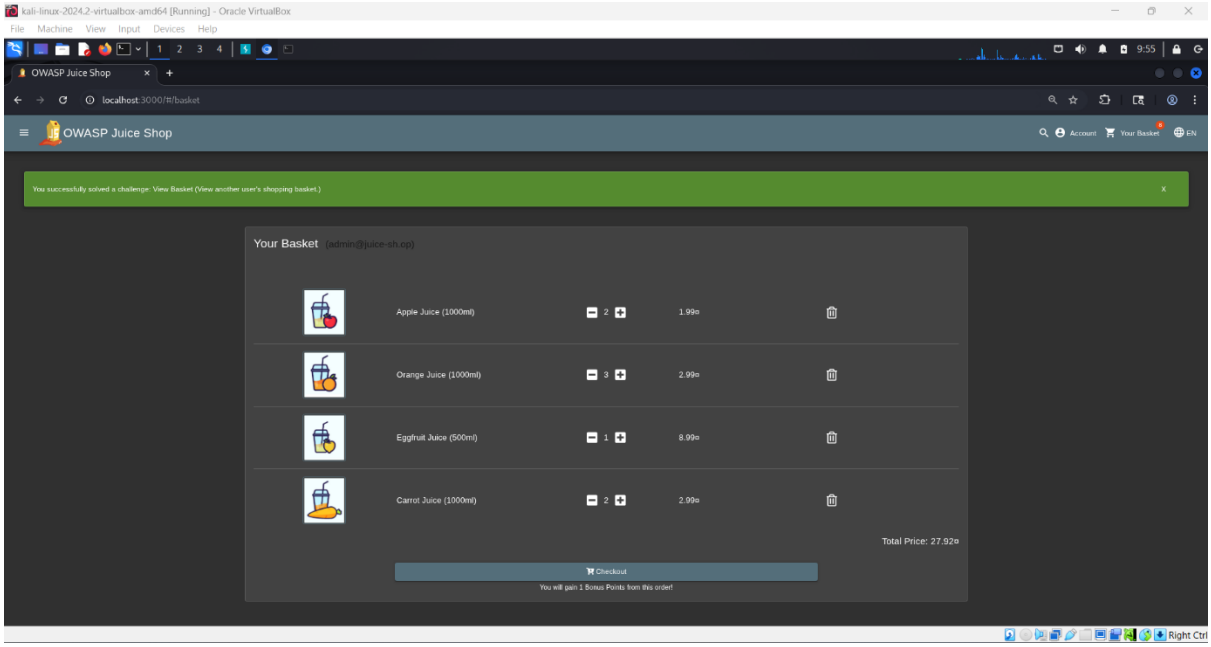
Steps to Reproduce

1. Open the application in a web browser and log in with a valid user account.
2. Configure an intercepting proxy tool like Burp Suite and ensure that traffic from the browser is being captured.
3. Perform an action that loads your basket like viewing your shopping cart or adding something to your cart.
4. In Burp Suite, intercept the outgoing HTTP request similar to following request:
   GET /rest/basket/6
5. Observe that the number at the end of the request (6) represents the user's basket ID.
6. Modify this ID to another value like changing 6 to 5:
   GET /rest/basket/5
7. Forward the modified request to the server.
8. Observe the response returned by the application.

Proof of Concept

The application processes modified object identifiers, indicating insufficient validation of user access.





Impact

- Unauthorized access to user resources
- Potential data leakage

Severity

High

Real-World Attack Scenario

- An attacker could manipulate object IDs in API requests to access other users' data, such as orders or account details. This could lead to unauthorized data exposure and privacy violations.

Mitigation

- Enforce proper authorization checks
- Validate user ownership of resources
- Use indirect references (e.g., UUIDs)

4.4Weak Authentication Mechanism

Description

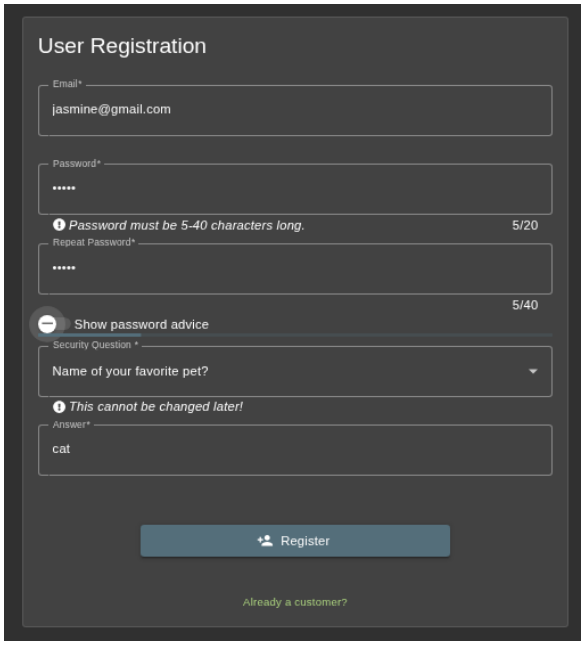
The application allows users to register with weak passwords, indicating lack of password policy enforcement.

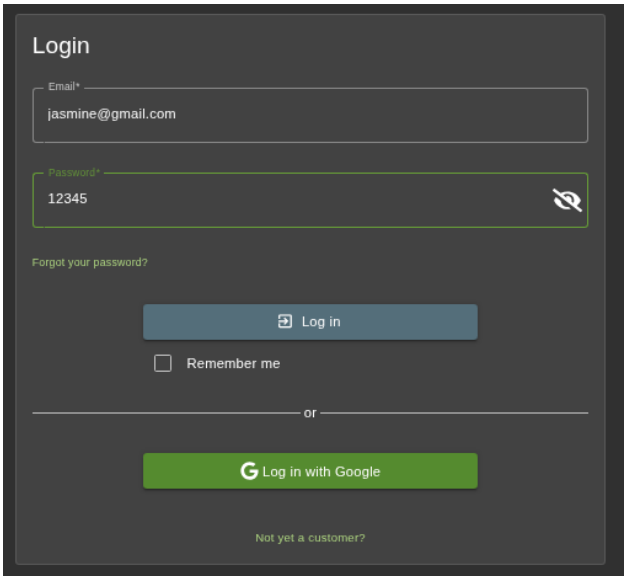
Steps to Reproduce

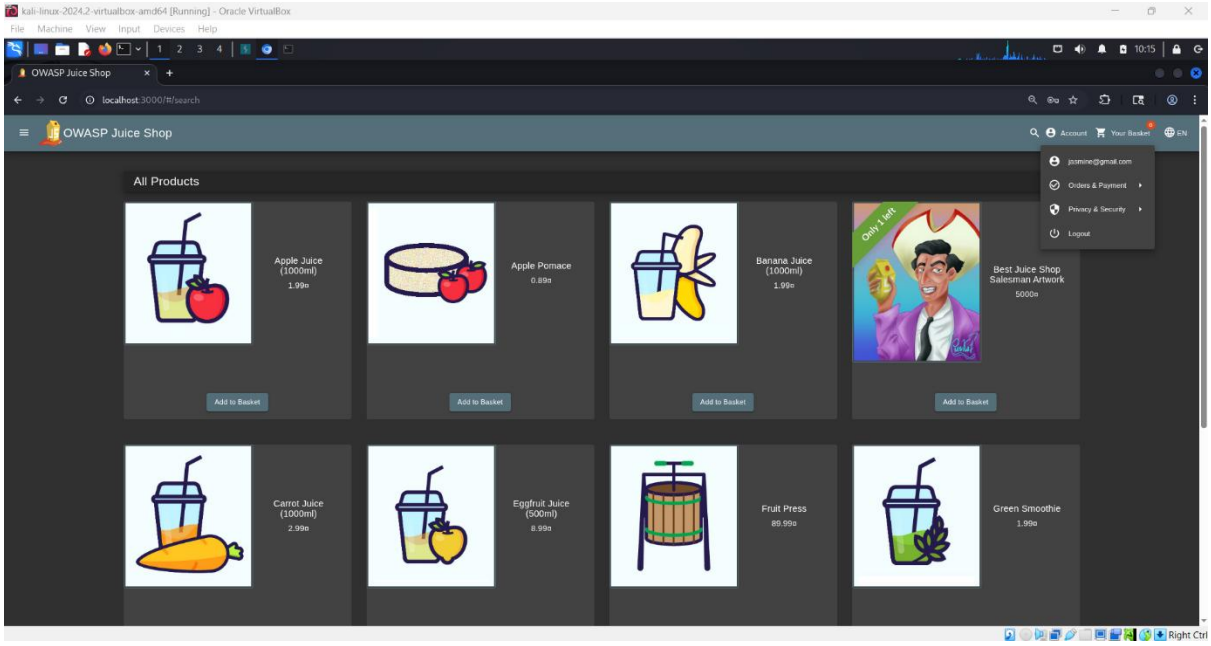
1. Open a web browser and navigate to the application.
2. Go to the registration page by clicking “Not yet a customer?” in the login page.
3. Fill the required registration details like email address.
4. In the Password field enter a weak password such as:
   12345
5. Confirm the same password in the Confirm Password field.
6. Submit the registration by clicking Register button.
7. Observe the behavior of the application after submission.

Proof of Concept

The registration is successful despite using a very weak password. That means the application accepts the weak password without validation or the application does not enforce a strong password policy.







Impact

- Increased risk of brute-force attacks
- User account compromise

Severity

Medium

Real-World Attack Scenario

- An attacker could exploit weak password policies by using common passwords (e.g., “12345”) to perform brute-force attacks and gain access to user accounts, leading to account takeover.

Mitigation

- Enforce strong password policies
- Require minimum length and complexity
- Implement multi-factor authentication

4.5Security Misconfiguration

Description

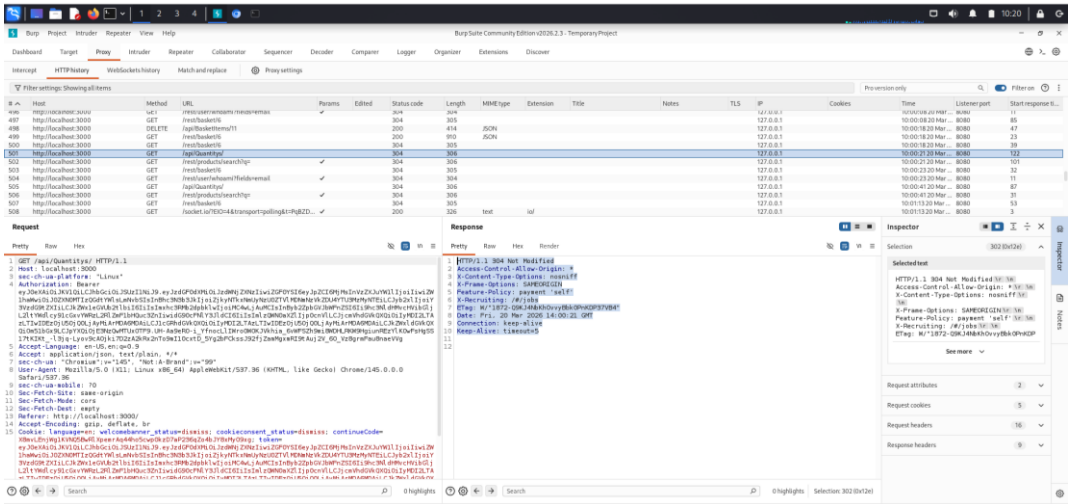
The application is affected by improper security configurations, including overly permissive CORS policies and missing security headers.

Steps to Reproduce

1. Open a web browser and navigate to the target application.
2. Configure an intercepting proxy tool like Burp Suite and ensure that HTTP/HTTPS traffic is being captured.
3. Perform any action in the application like loading the home page or logging in.
4. In Burp Suite, locate the captured HTTP response in the Proxy, HTTP History tab.
5. Select a request and inspect the corresponding HTTP response headers.

Proof of Concept

- Access-Control-Allow-Origin: * allows request from any origin
- Missing headers such as:
    - Content-Security-Policy
    - Strict-Transport-Security
- Presence of unnecessary headers exposing internal information



Impact

- Increased risk of cross-origin attacks
- Easier exploitation of XSS
- Exposure of internal application details

Severity

Medium

Real-World Attack Scenario

- An attacker could exploit misconfigured security settings, such as permissive CORS policies, to perform cross-origin attacks. This could allow malicious websites to interact with the application and access sensitive data without user consent.

Mitigation

- Restrict CORS policies
- Implement proper security headers
- Remove unnecessary response headers

4.6Sensitive Data Exposure (JWT Token)

Description

Sensitive user information is stored within JSON Web Tokens (JWT), which are accessible and can be decoded.

Steps to Reproduce

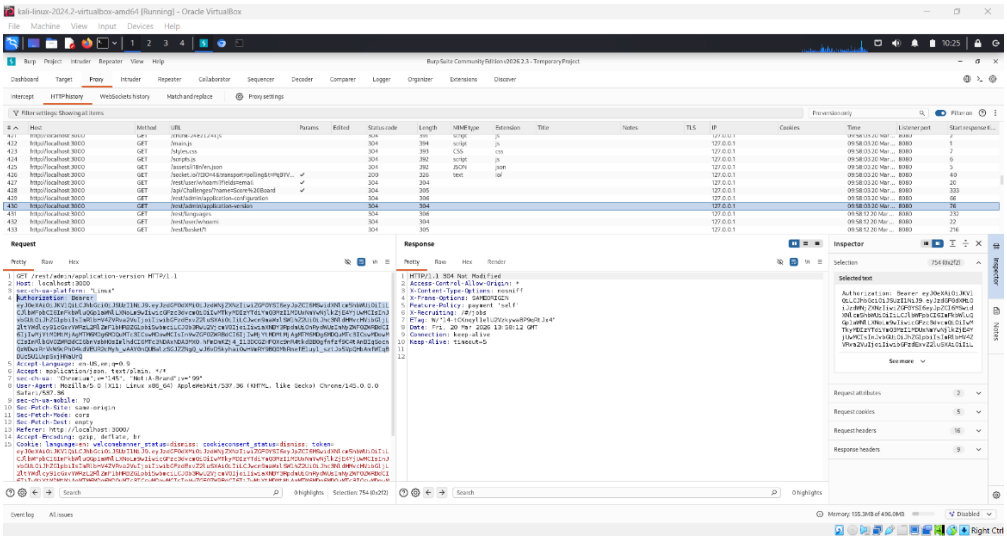
1. Open the application in a web browser and log in with a valid user account.
2. Configure an intercepting proxy tool such as Burp Suite and ensure that HTTP/HTTPS traffic is being captured.
3. Perform an action that sends an authenticated request like login, access profile, or view dashboard.
4. In Burp Suite, go to the Proxy, HTTP History tab and locate a request that contains an Authorization header or a token in cookies.
5. Identify the JWT token in the request. It is typically found in the Authorization header:

   Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
6. Copy the full JWT token value.
7. Navigate to an online TWT decoding tool like jwt.io.
8. Paste the copied token into the decoder input filed.
9. Observe the decoded output, which typically includes:
   - Header – algorithm, token type
   - Payload – user data like ID, email, roles

Proof of Concept

The decoded token reveals:

- User email
- User role
- User ID




```
JSON  Claims Breakdown

{
  "status": "success",
  "data": {
    "id": 1,
    "username": "",
    "email": "admin@juice-sh.op",
    "password": "0192023a7bbd73250516f069df18b500",
    "role": "admin",
    "deluxeToken": "",
    "lastLoginIp": "",
    "profileImage": "assets/public/images/uploads/defaultAdmin.png",
    "totpSecret": "",
    "isActive": true,
    "createdAt": "2026-03-20 13:08:04.177 +00:00",
    "updatedAt": "2026-03-20 13:08:04.177 +00:00",
    "deletedAt": null
  },
  "iat": 1774014071
}
```


Impact

- Session hijacking
- User impersonation
- Unauthorized access

Severity

High

Real-World Attack Scenario

- An attacker who gains access to a user's JWT token (e.g. via XSS or network interception) could decode it to extract sensitive information and reuse it to impersonate the user, gaining unauthorized access to the application.

Mitigation

- Avoid storing sensitive data in JWT payloads
- Use secure cookie flags (HttpOnly, Secure)
- Implement short token expiration

4.7Broken Access Control and Information Disclosure

Description

The application exposes administrative configuration data through an API endpoint without enforcing proper authorization checks. Additionally, caching behavior using the *If-None-Match* header can prevent the response from returning full data unless modified.

Steps to Reproduce

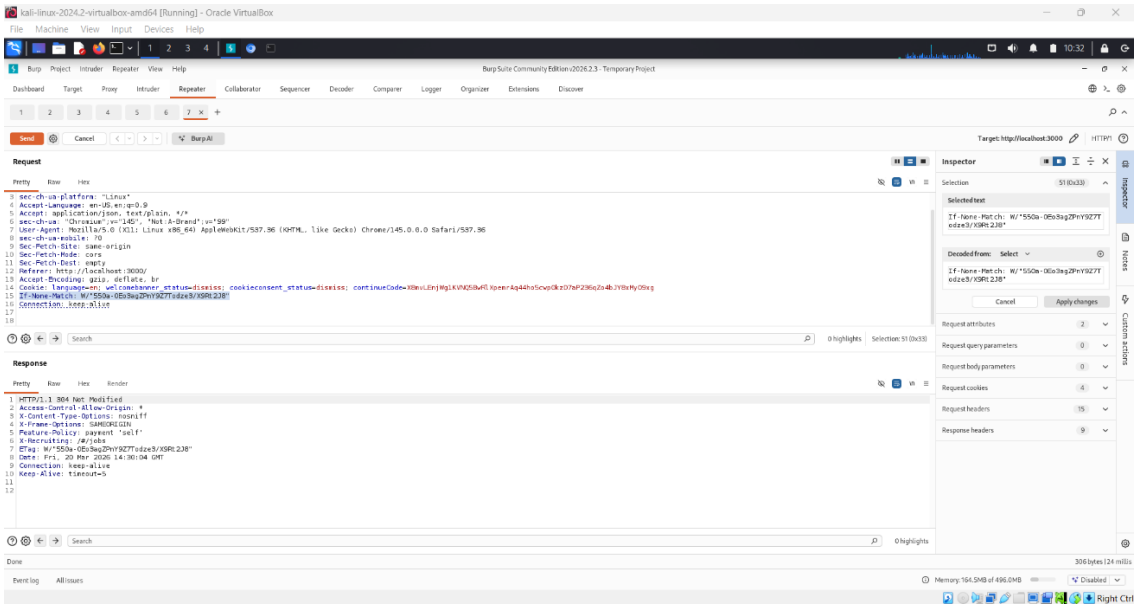
1. Intercept the request using Burp Suite
2. Send the following request:
   GET /rest/admin/application-configuration
3. Observe that the response may return 304 Not Modified due to the presence of the If-None-Match header
4. Remove the following header from the request:
   If-None-Match: W/"550a-OEo3agZPnY9Z7Todze3/X9Rt 2J8"
5. Resend the request
6. Analyze the response

Proof of Concept

After removing the If-None-Match header, the application returns full configuration data, including:

- Server configuration
- OAuth client ID
- Internal endpoints
- Application settings

This demonstrates that sensitive administrative data can be accessed without proper authorization controls.



Request

Pretty Raw Hex

```
1  GET /rest/admin/application-configuration HTTP/1.1
2  Host: localhost:3000
3  sec-ch-ua-platform: "Linux"
4  Accept-Language: en-US,en;q=0.9
5  Accept: application/json, text/plain, */*
6  sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
7  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
8  sec-ch-ua-mobile: ?0
9  Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3000/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=X8mvLEnjWglKVNQ5BwRlXpemrAq44ho5cwp0kzD7aP236qZo4bJY8xMyO9xg
15 Content-Length: 26
16
17 Connection: keep-alive
18
19
```

Request

Response

Pretty Raw Hex Render


```
            "message":"This website uses fruit cookies to ensure you get the juiciest tracking experience.",
            "dismissText":"Me want it!",
            "linkText":"But me wait!",
            "linkUrl":"https://www.youtube.com/watch?v=9PnbKL3wuH4"
        },
        "securityTxt":{
            "contact":"mailto:donotreply@owasp-juice.shop",
            "encryption":"https://keybase.io/bkimminich/pgp_keys.asc?fingerprint=19c01cb7157e4645e9e2c863062a85a8cbfbdcda",
            "acknowledgements":"/#/score-board",
            "hiring":"/#/jobs",
            "csaf":"/.well-known/csaf/provider-metadata.json"
        },
        "promotion":{
            "video":"owasp_promo.mp4",
            "subtitles":"owasp_promo.vtt"
        },
        "easterEggPlanet":{
            "name":"Orangeuze",
            "overlayMap":"orangemap2k.jpg"
        },
        "googleOauth":{
            "clientId":"1005568560502-6hm16lef8oh46hr2d98vf2ohlnj4nfhq.apps.googleusercontent.com",
            "authorizedRedirects":[
                {
                    "uri":"https://demo.owasp-juice.shop"
                },
                {
                    "uri":"https://juice-shop.herokuapp.com"
                },
                {
                    "uri":"https://preview.owasp-juice.shop"
                },
                {
                    "uri":"https://juice-shop-staging.herokuapp.com"
                },
                {
                    "uri":"https://juice-shop.wtf"
                },
```


Impact

- Exposure of internal system architecture
- Increased attack surface
- Potential misuse of configuration data

Severity

High / Critical

Real-World Attack Scenario

- An attacker could directly access administrative API endpoints without proper authorization and retrieve sensitive configuration data. This information could be used to understand the system architecture and launch further targeted attacks.

Mitigation

- Enforce strict authorization checks on all admin endpoints
- Do not rely on client-controlled headers for security
- Restrict access to sensitive API responses
- Implement proper server-side access validation

7. Conclusion

The security assessment of the OWASP Juice Shop application revealed multiple vulnerabilities across different areas, including input validation, authentication, authorization, and system configuration. The identified issues range from critical vulnerabilities such as SQL Injection and Broken Access Control to medium-level risks like security misconfiguration and weak authentication mechanisms.

These vulnerabilities demonstrate how improper handling of user input, lack of access control enforcement, and insecure configurations can significantly impact the overall security of a web application. In a real-world scenario, such weaknesses could allow attackers to gain unauthorized access, execute malicious scripts, and expose sensitive data.

The assessment highlights the importance of integrating security practices throughout the software development lifecycle to prevent such vulnerabilities.

8. Recommendations

Based on the findings, the following recommendations are suggested to improve the security posture of the application:

1. **Input Validation & Output Handling**
   - Implement strict server-side input validation
   - Use parameterized queries to prevent SQL Injection
   - Encode all outputs to prevent Cross-Site Scripting (XSS)

2. **Authentication & Authorization**
   - Enforce strong password policies
   - Implement multi-factor authentication (MFA)
   - Apply proper role-based access control (RBAC)
   - Validate user permissions for every request

3. **Secure Configuration**
   - Restrict CORS policies to trusted origins only
   - Add missing security headers:
     - Content-Security-Policy
     - Strict-Transport-Security
     - X-Content-Type-Options
   - Remove unnecessary or sensitive headers

4. **Sensitive Data Protection**
   - Avoid storing sensitive information in JWT tokens
   - Use secure cookies with HttpOnly and Secure flags
   - Implement short token expiration and refresh mechanisms

5. **General Security Practices**
   - Enforce HTTPS for all communications
   - Perform regular security testing and code reviews
   - Keep dependencies and frameworks up to date
   - Follow OWASP Top 10 guidelines during development